# Advanced Computing Architectures

PROF. ROBERTO PASSERONE
DISI, UNIVERSITY OF TRENTO
AY 2025/2026

# Welcome to class

**Advanced Computing Architectures**

- Master's degree in Information Engineering
- Master's degree in Computer Science
- 48 hours, 6 credits

**Lecturer**

- Roberto Passerone
  - Tel: 0461 28-3971
  - E-mail: roberto.passerone@unitn.it
  - Office: First floor, Povo 2
  - Office hours by appointment

# Schedule

| | Monday | Tuesday | Wednesday | Thursday | Friday |
|---|---|---|---|---|---|
| 08:30 – 09:30 | | | | | |
| 09:30 – 10:30 | | | | | |
| 10:30 – 11:30 | | | | | |
| 11:30 – 12:30 | B108 | | A220 | | |
| 12:30 – 13:30 | B108 | | A220 | | |
| 13:30 – 14:30 | | | | | |
| 14:30 – 15:30 | | | | | |
| 15:30 – 16:30 | | | | | |
| 16:30 – 17:30 | | | | | |
| 17:30 – 18:30 | | | | | |
| 18:30 – 19:30 | | | | | |

# STUDY MATERIAL

**Slides**

- Available on the course web site
- Use the facilities offered by ESSE3

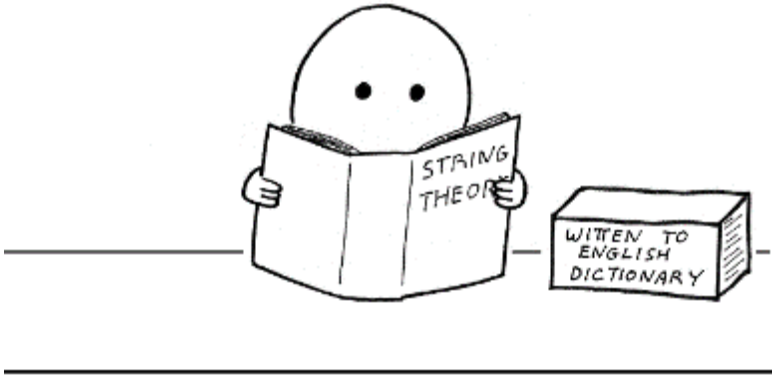
**Books**

- J. H. Hennessy, D. A. Patterson, "Computer Architecture: a quantitative approach," Morgan Kaufmmann. Various editions.
- J. Cheng, M. Grossman, T. McKercher, “Professional CUDA C Programming,” John Wiley & Sons

# Pre-requisites

**The following topics are assumed**

- Basic computer architectures
  - What is a register, memory, CPU, etc.
- Knowledge of C/C++ or at least one classic programming language
- Plus: assembly language programming



# Evaluation

**Written exam**

- Exercises and open questions
- Evaluation from 0 to 30

**Oral exam**

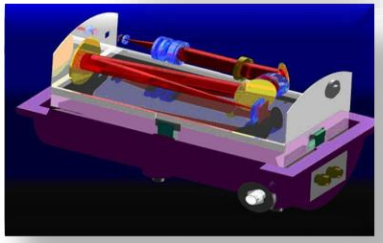
- Optional
- Only if written is at least 15/30
- Adds or subtracts 3 to written exam ([-3, +3])

**Mid-term**

- Not scheduled

**Long Project**

- Optional, replaces the exam
- Example: Develop an advanced microprocessor
- Example: GPU programming
- Example: Algorithm acceleration (FPGA, GPU)



# Outline of the course

1. **Fundamentals and metrics**
2. **Instruction Set Architectures**
3. **Pipelining**
4. **Memory organization**
5. **Instruction-Level Parallelism**
    1. Branch prediction
    2. Dynamic scheduling
    3. Speculative execution, superscalar
6. **Thread-level Parallelism**
7. **VLIW architectures, in-memory processing**
8. **Data-level Parallelism, GPUs**
    1. Programming laboratory
9. **Multiprocessors**

# Course contents and objectives

## Qualitative

**Understand the working principle of most recent computing devices**

- Operating principles of caches and predictors
- Instruction scheduling
- Parallelism: Superscalar vs. Very Long Instruction Word (VLIW) vs. data parallelism

# Course contents and objectives

## Quantitative

**Analyze the fundamental design criteria of systems based on processing elements**

- Cost, **performance** and power trade-offs at the architectural level
- Estimation of performance metrics

# Course contents and objectives

## Objective

**Acquire an adequate knowledge of low-level processing architectures in order to design microprocessor-based systems**

**Use knowledge of low-level processing architectures to develop better software**

# Laboratory

- **Will spend a fair amount of time on GPU programming**
  - Learn about the architecture
  - Learn the programming and execution model
  - Understand what affects performance
- **Develop simple kernels**
  - Use CUDA as the underlying model
  - Develop several procedures for vector and matrix operations
  - Compile, execute and evaluate on a server

# But first

- **Need to understand ways to measure and quantify performance**
  - Trickier than might seem at first sight
  - Fundamental to direct efforts to best solutions
- **Various metrics**
  - Cost – primary economic concern
  - Performance – want systems that compute quickly
  - Power – increasingly important for mobile devices

## METRICS

### POWER AND ENERGY ARE EXPRESSED IN *WATTS* AND *JOULES*, RESPECTIVELY

# POWER CONSUMPTION EVALUATION

Power = Energy per unit time

# Trend

**In CMOS technology the power consumption is due to**

- Dynamic (or Active): currents related to
  - charging/discharging of logic gates during switching
  - transient short-circuits during switching
- Static: leakage (sub-threshold) currents in transistors

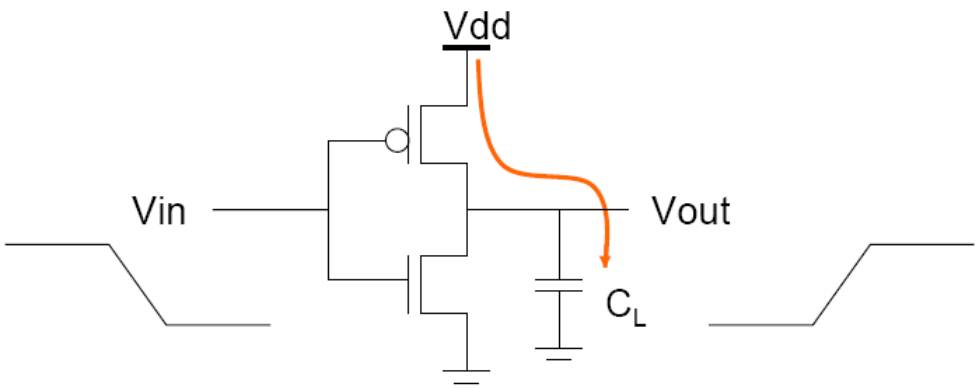
**According to the current trend, with each process generation**

- Supply voltage will scale by only ~0.8
- Dynamic power will scale by ~0.9, but power density will increase by ~30-80%
- Static power will increase at an exponential rate

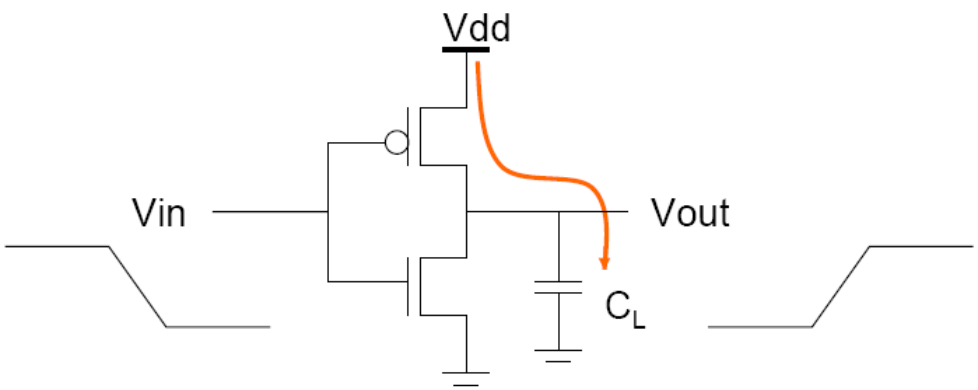
# Dynamic power consumption - 1

## Charging and discharging

- Dynamic power consumption is mostly due to charging and discharging of the capacitive load of every logic gate inside a digital CMOS integrated circuit



# Dynamic power consumption - 1




**Capacitance: Function of wire length, transistor size**

**Supply Voltage: Has been dropping with successive fab generations**

**Switching probability: depends on signal statistics**

$$\text{Energy/transition} = C_L * V_{DD}^2 * P_{0\to1}$$

$f_{0\to1}$

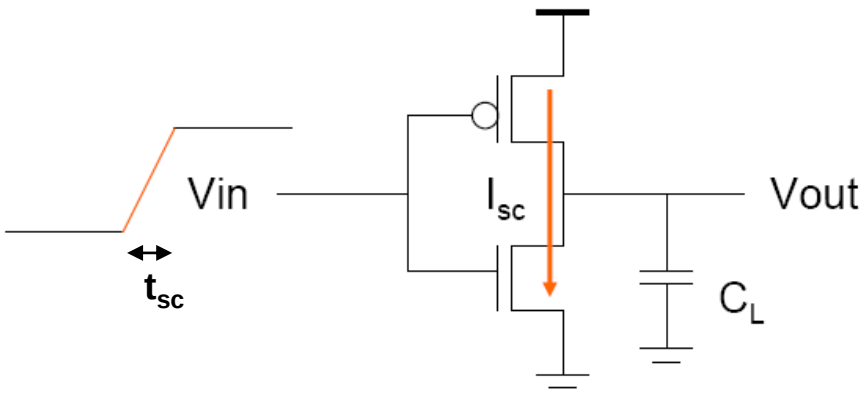
$$\text{Power} = \text{Energy/transition} * f = C_L * V_{DD}^2 * P_{0\to1} * f$$

**Clock frequency: increases…**

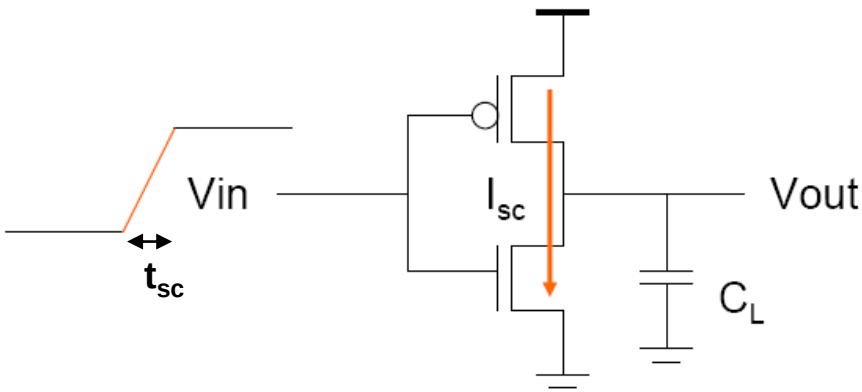
# Dynamic power consumption - 2

**Short circuit current**

- Finite slope of the input signal causes a direct current path between VDD and GND for a short period of time during switching when both the nMOS and pMOS transistors are conducting



# Dynamic power consumption - 2



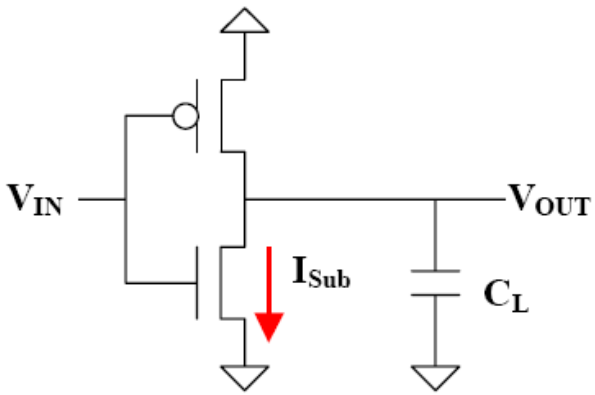

Duration of the rising/falling time of the input signal

$$E_{sc} = t_{sc}\, V_{DD}\, I_{peak}\, P_{0\to1}$$

$$P_{sc} = t_{sc}\, V_{DD}\, I_{peak}\, f_{0\to1}$$

$I_{peak}$ determined by the saturation current of the P and N transistors (which depends on their sizes, process technology, temperature, etc….) and the ratio between input and output slopes

# Static power consumption



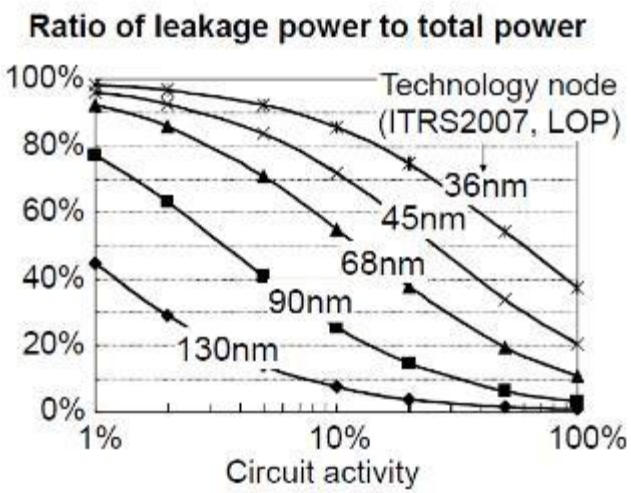

$$I_{DSub} = k \cdot e^{\frac{-q \cdot V_T}{a \cdot k_a \cdot T}}$$

Threshold voltage

Temperature

Technology-dependent constants

- Sub-threshold current grows exponentially with temperature and with decreasing threshold voltage
- Continued scaling of supply voltage will make sub-threshold conduction the primary source of power dissipation



# Summing up

$$P = C_L V_{DD}^2 P_{0\to1} f + t_{sc} V_{DD} P_{0\to1} I_{peak} f + V_{DD} I_{leakage}$$

**Equation can be used to estimate the power consumption**

- However, determining the individual parameters is not easy

**Parameters under architectural control**

- $\mathbf{f_{clock}}$ affects performance
- $\mathbf{C_L}$ depends on the number of gates
- $\mathbf{V_{DD}}$ affects performance
- $\mathbf{P_{0\to1}}$ depends on the implementation and architecture

## METRIC

## COST IS EXPRESSED IN US DOLLARS

# COST ESTIMATION

# TREND

**Device cost tends to decrease over time even without improvements in the basic implementation technology (learning curve)**

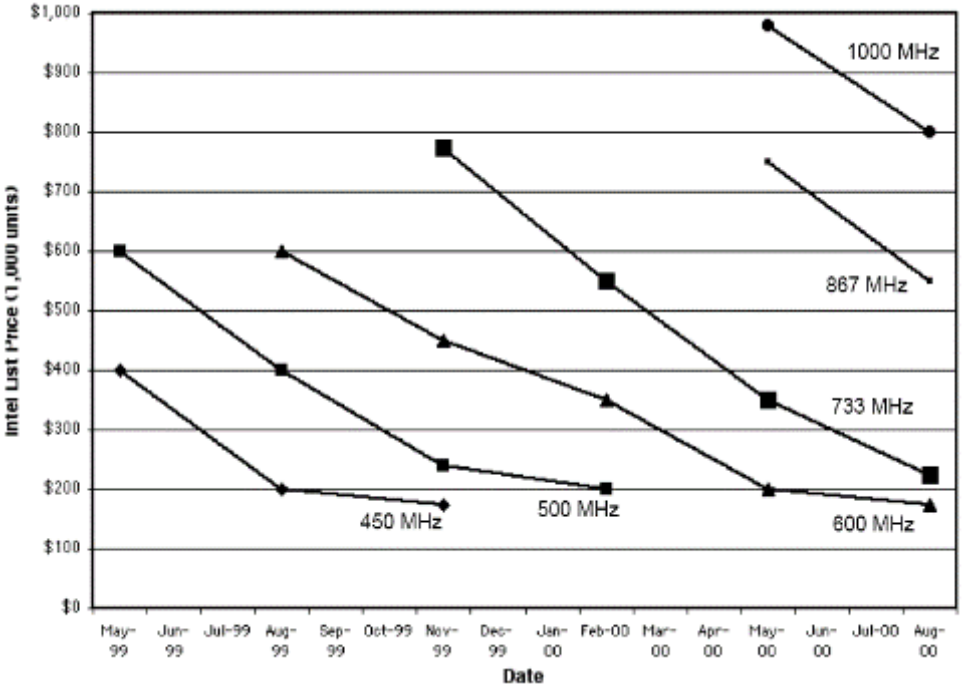
- Better yield: larger percentage of devices which survive testing

**By increasing volumes, costs decrease (~10% doubling production):**

- Faster learning curve
- Better efficiency
- Non-recursive engineering (NRE) and development costs amortized over many machines

**Commodity market effect**

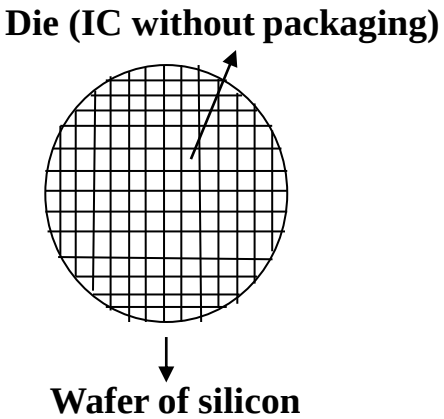
- Manufacturing costs and selling prices are close due to large competition




**Example: the price of an Intel Pentium III at a given frequency decreases over time as yield enhancements decrease the cost of good die and competition forces price reductions**

# Cost of an integrated circuit - 1

**Experimentally, the cost of an integrated circuit is given by:**

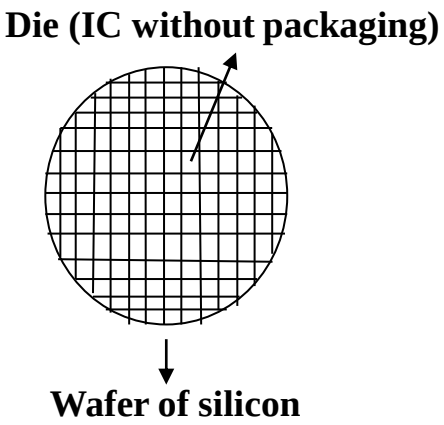



$$C = \frac{C_{die} + C_{testing} + C_{packaging}}{Y}$$

**where**

- $C_{die}$ is the cost of manufacturing a die
- $C_{testing}$ is the cost for testing a die after cutting the wafer of silicon into individual dies
- $C_{packaging}$ is the cost for packaging and then testing the working dies
- $Y$ is the yield after the final test (percent of working chips)

# Cost of an integrated circuit - 1




$$C = \frac{C_{die} + C_{testing} + C_{packaging}}{Y}$$

- $C_{testing}$ and $C_{packaging}$ are fixed once the IC is made
- They are mostly a function of the complexity of the IC and NOT of the manufacturing process

**On the contrary:**

$$C_{die} = \frac{C_{wafer}}{N_{die} \cdot Y_{die}}$$

Cost of making a wafer

Total number of dies per wafer

Die yield (before packaging)

# Cost of an integrated circuit - 2

$$N_{die} = \frac{\pi \cdot D^2/4}{A_{die}} - \frac{\pi \cdot D}{\sqrt{2 \cdot A_{die}}}$$

**Total number of dies on a round wafer**

**Number of dies close to the periphery of the wafer**

- Strong dependence of $C$ on $A_{die}$

**where $D$ is the diameter of the wafer and $A_{die}$ is the area of the die**

$$Y_{die} = Y_{wafer} \cdot \left(1 + \frac{N_{defects} \cdot A_{die}}{\alpha}\right)^{-\alpha}$$

**where**

- $Y_{wafer}$ is the wafer yield (today close to 100%)
- $N_{defects}$ is the number of defects of the wafer per unit area (e.g., 0.4 - 0.8 defects/cm$^2$)
- α is an empirical value related to the complexity of the manufacturing process (today ~4.0)

- **$A_{die}$ is the only parameter a designer can control.**
- **All the others depend on the manufacturing process!**

METRICS
CLOCK RATE
MILLIONS OF INSTRUCTIONS PER SECOND (MIPS)
MILLIONS OF INTEGER OPERATION OR FLOATING POINT OPERATIONS PER SECOND (MOPS OR MFLOPS)
EXECUTION TIME

# PERFORMANCE EVALUATION

# TREND

**Performance evolution of top processors takes advantage of Moore's Law**

- Clock frequencies double every 15-18 months

**Performance grows much faster**

- Effective enhancement are applied not only at the technology level (transistor scaling), but also at the architectural level (higher parallelism)

**Accurate trend estimation hard**

- Lack of univocal testing criteria
- Use of misleading performance metrics

**Basic question: "How much is X faster than Y?"**

# Ideal properties of a performance metric

**Linearity**

- a metric is linear if when the values of the metric differ by a certain factor, also the actual performance of the machines differs by the same factor

**Reliability**

- a performance metric is reliable if a system X outperforms a system Y when the corresponding values of the metric for both systems indicate that X should outperform Y

**Repeatability**

- a performance metric is repeatable if subsequent measurements return compatible results (within a given amount of uncertainty)

**Easiness of use**

- if a metric is difficult to use, few people will use it. Also, the probability to obtain incorrect results grows

**Consistency**

- A consistent metric is one for which the units of the metric and its precise definition are the same across different systems and configurations

**Independence**

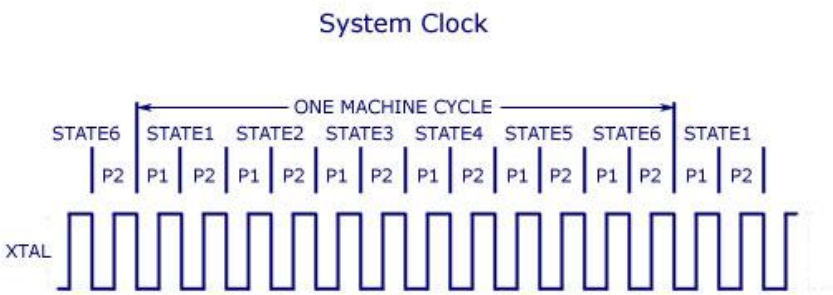
- a performance metric is independent if its definition is univocal (i.e., not affected by custom external influences)

# The clock rate

**The clock rate of a CPU is a very misleading metric, introduced by microprocessor companies mostly for marketing reasons**

**This metric is:**

- Repeatable (it is a constant)
- Easy to measure
- Consistent
- Independent




**but it is NOT:**

- Linear
- Reliable

**The clock rate does not take the amount of computation within a clock cycle into account**

**It ignores the interaction of the CPU with the memory and I/O systems**

**Example: An AMD Athlon K7 running at 1.8 GHz proved to be considerably faster than an Intel Celeron running at 2.0 GHz**

# MIPS

**Million of Instructions Per Second**

$$MIPS = \frac{N}{T \cdot 10^6}$$

**Total number of instruction to be executed**

**Time to execute the N instructions**

```
xor eax,eax
mov frametimer,eax      //Zero out frametimer.
mov frameid,1           //Open your eye.

mov damage,~99          //Deal 99 damage.
cmp frametimer,7        //A very short blink rate.
jl :do_not_open_eye
mov frameid,2
add yvel,60             //Acceleration in the y-direction
cmp yvel,600
jle :do_not_limit
mov yvel,600            //Limit the Y velocity to a maximum value.
and collision,8         //Check one bit of the collision.
                        //Have we hit the bottom wall?
jz :velocity_loader     //--- We haven't hit the wall
mov scriptstate,2       //--- We have hit the wall.

push 1
push 48                 //0x48 = Crashy-puffy sound.
call 00420640           //Play sound effect.
add esp,8
mov edx,[ebp+8]

push 0                  //Initial slot.
push 0                  //We don't need a child entity.
push 0                  //Direction of smoke
```

**This metric is:**

- Repeatable
- Easy to measure
- Independent

**but it is NOT:**

- Linear: ignores the amount of computation carried out by any single instruction
- Reliable: same as above, could be reliable if comparing processors with similar ISA
- Consistent: the meaning of “instruction” changes between different architectures

# MOPS AND MFLOPS

**MOPS and MFLOPS consider only integer and floating-point mathematical operations**

- try to solve the problem of MIPS by virtue of the better consistency of the definition of mathematical operation with respect to the more vague concept of instruction

**Total number of integer ops to be executed**

$$MOPS = \frac{O}{T \cdot 10^6}$$

**Time to execute the *O* integer ops**

**Total number of fp ops to be executed**

$$MFLOPS = \frac{F}{T \cdot 10^6}$$

**Time to execute the *F* floating point ops**

# MOPS AND MFLOPS

**This attempt was unfruitful**

- MOPS and MFLOPS are still unreliable and inconsistent, because they do not consider how the mathematical operations are implemented
- Example: if a processor X executes a multiplication as a sequence of additions, it carries out a larger number of ops than a processor Y with a HW multiplier
- As a consequence, the values of MOPS or MFLOPS associated to X will be larger than those of Y, even though X is slower than Y.

**Total number of integer ops to be executed**

$$MOPS = \frac{O}{T \cdot 10^6}$$

**Time to execute the *O* integer ops**

**Total number of fp ops to be executed**

$$MFLOPS = \frac{F}{T \cdot 10^6}$$

**Time to execute the *F* floating point ops**

# Execution time

**The execution time is the time required to complete a task running on a computing device, including all the accessory operations necessary to this purpose (e.g. disk access, memory operations and so on…)**

- The performance of a system is inversely proportional to execution time

**Execution time is the only metric meeting all the properties**

- Linearity: doubling performance means halving execution time
- Reliability: if the execution time of a program running on X is lower than Y, certainly X is faster than Y
- Repeatability: when repeating measurements, compatible values will result, the random fluctuations being caused by occasional system variations
- Easiness to measure: execution time of an application or of a part of a program can be easily measured using OS functions or commands which access the counters on chip
- Consistency: the unit of measurement is the second
- Independence: the definition of the metric is univocal and unambiguous

# Execution time vs. CPU time

**Execution time includes**

- The time spent in running a certain task on the CPU (CPU time)
- The waiting time related to memory and I/O operations
- The time spent running other tasks before the measured task is finished (in a multitasking environment)

**CPU time is a more objective indicator of the effective performance of the CPU without considering other system operations**

**CPU time consists of two contributions**

- ***User CPU time:*** the time spent when executing the instructions of the task to be tested
- ***System CPU time:*** the time spent when executing the OS instructions (if an OS is present) which are required by the task to be tested

# Execution time vs. CPU time

**Example**

- In UNIX the shell command *time* applied to a certain task returns

```
90.7u 12.9s 2:39 65%
```

- 90.7u – User CPU time
- 12.9s – System CPU time
- 2:39 – actual total wall clock execution time
- 65% – percentage of CPU time over wall clock

  (90.7 + 12.9) / (2*60 + 39) = 65%

# THE CPU PERFORMANCE EQUATION

**CPU time is computed through the CPU performance equation**

$$CPU\ time = N_{CLK} \cdot T_{CLK} = CPI \cdot IC \cdot T_{CLK}$$

**where:**

- $N_{CLK}$ is the **number of clock cycles** associated to the task
- $T_{CLK} = 1 / F_{CLK}$ is the **clock period**
- $IC$ represents the **Instruction Count** of the task considered
- $CPI = N_{CLK} / IC$ is the **Clock Per Instruction**, a parameter describing the average duration of a single instruction
- Sometimes, the reciprocal value, i.e., the Instructions per Clock ($IPC = 1 / CPI$) is used.

# The CPU performance equation

**Let**

- *n* be the total number of instructions of the ISA of a certain processor
- $IC_i$ and $CPI_i$ represent the instruction count and the CPI of the i-th instruction

**then**

Relative frequency of instruction *i*

$$CPI = \frac{\sum_{i=1}^{n} IC_i \cdot CPI_i}{IC}$$

**The CPI (or IPC) is a very important design parameter because it is a metric to estimate the performance of the ISA and the organization of a certain architecture, regardless of the underlying hardware technology (e.g., the achievable clock rate)**

# Benchmarks

**Execution time is measured over the execution of a task**

- Which task should we execute?
- To compare the performance of different systems, we need to run suitable applications, which are the same across multiple computers and that mirror the expected “workload” for a given computing system

**The suite of tasks used to measure performance is called a *benchmark***

# Types of Benchmarks

1. **Real Applications**
   - real working applications matching the average workload of a certain system. They may have portability problems, and the repeatability can be poor due the large number of employed options
2. **Modified (scripted) Applications**
   - the same as before with some minor modification (e.g., input stimulus script, I/O removal) to enhance portability and to focus on one aspect of system performance
3. **Kernels**
   - small, key sections of real application extracted to evaluate the performance of individual features
4. **Synthetic benchmarks (e.g., Dhrystone, Whetstone)**
   - similar to kernels, but without any practical usefulness. They try to match the average execution frequency of the main instructions included in the expected workload
5. **Toy benchmarks**
   - 10-100 lines of code, running simple and very portable algorithms, whose solution is known (e.g., Quicksort, puzzle, …)

# Benchmark selection

**Problem: companies tend to select and optimize benchmark applications to highlight the performance of their products**

**To address this problem:**

- Suites of multiple applications are used. In this way, the weakness and the specific features of any benchmark are lessened by those of the others
- Benchmark source code modifications generally are not allowed
- System configuration and compiler options have to be reported along with the measured data. Sometimes, the type of compiler and its options are set appropriately → baseline performance results

**Depending on the computing classes different kinds of benchmarks are defined**

- ***Desktop computing***: the most famous benchmark suite is defined by the Standard Performance Evaluation Corporation (SPEC)
- ***Servers***: benchmark suite defined by SPEC and Transaction Policy Council (TPC)
- ***Embedded computing***: the benchmark suite is defined by the EDN Embedded Microprocessor Benchmark Consortium (EEMBC)

# THE SPEC2006 BENCHMARK SUITE

**The SPEC suite consists of real applications**

**Suite changed over the years (SPEC89, SPEC92, SPEC95, SPEC2000, SPEC2006) due to**

- program obsolescence
- excessive sensitivity to optimizations (too much cheating)

**SPEC 2006 consists of**

- **12 integer benchmarks**
- **17 floating point benchmarks**

| | Benchmark name by SPEC generation | | | | |
|---|---|---|---|---|---|
| **SPEC2006 benchmark description** | **SPEC2006** | **SPEC2000** | **SPEC95** | **SPEC92** | **SPEC89** |
| GNU C compiler | | | | | gcc |
| Interpreted string processing | | | perl | | espresso |
| Combinatorial optimization | | mcf | | | li |
| Block-sorting compression | | bzip2 | | compress | eqntott |
| Go game (AI) | go | vortex | go | sc | |
| Video compression | h264avc | gzip | ijpeg | | |
| Games/path finding | astar | eon | m88ksim | | |
| Search gene sequence | hmmer | twolf | | | |
| Quantum computer simulation | libquantum | vortex | | | |
| Discrete event simulation library | omnetpp | vpr | | | |
| Chess game (AI) | sjeng | crafty | | | |
| XML parsing | xalancbmk | parser | | | |
| CFD/blast waves | bwaves | | | | fpppp |
| Numerical relativity | cactusADM | | | | tomcatv |
| Finite element code | calculix | | | | doduc |
| Differential equation solver framework | dealII | | | | nasa7 |
| Quantum chemistry | gamess | | | | spice |
| EM solver (freq/time domain) | GemsFDTD | | | swim | matrix300 |
| Scalable molecular dynamics (~NAMD) | gromacs | | apsi | hydro2d | |
| Lattice Boltzman method (fluid/air flow) | lbm | | mgrid | su2cor | |
| Large eddie simulation/turbulent CFD | LESlie3d | wupwise | applu | wave5 | |
| Lattice quantum chromodynamics | milc | apply | turb3d | | |
| Molecular dynamics | namd | galgel | | | |
| Image ray tracing | povray | mesa | | | |
| Spare linear algebra | soplex | art | | | |
| Speech recognition | sphinx3 | equake | | | |
| Quantum chemistry/object oriented | tonto | facerec | | | |
| Weather research and forecasting | wrf | ammp | | | |
| Magneto hydrodynamics (astrophysics) | zeusmp | lucas | | | |
| | | fma3d | | | |
| | | sixtrack | | | |

# SPEC 2017

| SPECrate 2017 Integer | SPECspeed 2017 Integer | Language [1] | KLOC [2] | Application Area |
|---|---|---|---|---|
| 500.perlbench_r | 600.perlbench_s | C | 362 | Perl interpreter |
| 502.gcc_r | 602.gcc_s | C | 1,304 | GNU C compiler |
| 505.mcf_r | 605.mcf_s | C | 3 | Route planning |
| 520.omnetpp_r | 620.omnetpp_s | C++ | 134 | Discrete Event simulation - computer network |
| 523.xalancbmk_r | 623.xalancbmk_s | C++ | 520 | XML to HTML conversion via XSLT |
| 525.x264_r | 625.x264_s | C | 96 | Video compression |
| 531.deepsjeng_r | 631.deepsjeng_s | C++ | 10 | Artificial Intelligence: alpha-beta tree search (Chess) |
| 541.leela_r | 641.leela_s | C++ | 21 | Artificial Intelligence: Monte Carlo tree search (Go) |
| 548.exchange2_r | 648.exchange2_s | Fortran | 1 | Artificial Intelligence: recursive solution generator (Sudoku) |
| 557.xz_r | 657.xz_s | C | 33 | General data compression |

| SPECrate 2017 Floating Point | SPECspeed 2017 Floating Point | Language [1] | KLOC [2] | Application Area |
|---|---|---|---|---|
| 503.bwaves_r | 603.bwaves_s | Fortran | 1 | Explosion modeling |
| 507.cactuBSSN_r | 607.cactuBSSN_s | C++, C, Fortran | 257 | Physics: relativity |
| 508.namd_r | | C++ | 8 | Molecular dynamics |
| 510.parest_r | | C++ | 427 | Biomedical imaging: optical tomography with finite elements |
| 511.povray_r | | C++, C | 170 | Ray tracing |
| 519.lbm_r | 619.lbm_s | C | 1 | Fluid dynamics |
| 521.wrf_r | 621.wrf_s | Fortran, C | 991 | Weather forecasting |
| 526.blender_r | | C++, C | 1,577 | 3D rendering and animation |
| 527.cam4_r | 627.cam4_s | Fortran, C | 407 | Atmosphere modeling |
| | 628.pop2_s | Fortran, C | 338 | Wide-scale ocean modeling (climate level) |
| 538.imagick_r | 638.imagick_s | C | 259 | Image manipulation |
| 544.nab_r | 644.nab_s | C | 24 | Molecular dynamics |
| 549.fotonik3d_r | 649.fotonik3d_s | Fortran | 14 | Computational Electromagnetics |
| 554.roms_r | 654.roms_s | Fortran | 210 | Regional ocean modeling |

Rate: throughput (e.g., jobs per hour)
Speed: time

# SPECbase vs. SPECpeak

**The SPEC consortium allows manufacturers to report two kinds of data records:**

- baseline performances (SPECbase), and
- peak performances (SPECpeak)

**Baseline restrictions**

- Only 4 compiler flags
- Same settings for all programs
- Can't provide program-specific info
- Same compiler for all runs

**Degrees of freedom to achieve peak performance**

- “Guaranteed never to exceed” level of performance
- Disable network and display
- Single user mode
- Different compile flags or compilers
- Can Modify OS kernel

# SPEC CPU PERFORMANCE INDEXES

**Question**

- **how do we summarize the relative performance related to a benchmark suite?**

**SPEC 2000 suggests 2 indexes:**

- **CINT2000 for integer benchmarks, and**
- **CFP2000 for floating point benchmarks.**

**Both indexes are obtained as the geometric mean of the normalized execution times of the programs of the suite**

- For SPEC2000 and SPEC2006, the reference machine is a Sun SPARC Ultra 10 @ 333 MHz
- For SPEC2017, the reference machine is a Sun Fire V490 with 2100 MHz UltraSPARC-IV+ chips

# SPEC CPU PERFORMANCE INDEXES

Number of programs

$$G = \sqrt[N]{\prod_{j=1}^{N} \frac{T_{e_j}}{T_{R_j}}}$$

Execution time of the *j*-th program

Reference execution time of the *j*-th program

Geometric mean of the normalized execution times relative to a Sun SPARC Ultra 10 @ 333 MHz

*Source: www.apple.com*

Note: the SPEC index is actually the **inverse** of the ratio above, given that the reference machine has an index of 100. So, the higher, the better!

# Pros and Cons of the geometric mean

**Advantage**

- The geometric mean maintains consistent relationships when comparing normalized values regardless of the reference machine

**Disadvantage**

- Long and short tasks have the same weight
- Engineers are encouraged to focus their attention on the benchmarks whose relative performance is easiest to improve rather than on the applications which are slowest

**Example**

- The effect of improving the execution time of a program P1 from 2 s to 1 s is the same as improving the execution time of a program P2 from 1000 s to 500 s. Obviously, the absolute increment in performance in the latter case is much larger, but SPEC does not consider it

# Pitfall in the use of geometric mean

**The geometric mean does not predict execution time**

- The metric is unreliable!

**Example**

- Consider two programs P1 and P2 running on 3 machines A, B and C
- C is faster than A and B, whereas A and B are comparable
- However, the total execution time on A and B depends on the frequency of execution of P1 and P2
- A and B take the same execution time only if P1 is executed 100 times more than P2

| | Normalized to A | | | Normalized to B | | | Normalized to C | | |
|---|---|---|---|---|---|---|---|---|---|
| | A | B | C | A | B | C | A | B | C |
| Program P1 | 1.0 | 10.0 | 20.0 | 0.1 | 1.0 | 2.0 | 0.05 | 0.5 | 1.0 |
| Program P2 | 1.0 | 0.1 | 0.02 | 10.0 | 1.0 | 0.2 | 50.0 | 5.0 | 1.0 |
| Geometric mean | 1.0 | 1.0 | 0.63 | 1.0 | 1.0 | 0.63 | 1.58 | 1.58 | 1.0 |

| | Computer A | Computer B | Computer C |
|---|---|---|---|
| Program P1 (secs) | 1 | 10 | 20 |
| Program P2 (secs) | 1000 | 100 | 20 |
| Total time (secs) | 1001 | 110 | 40 |

# Other criteria to summarize performance

**Total execution time**

- Obtained by adding up the execution times of the different programs of the benchmark suite

$$T = \sum_{j=1}^{N} T_{e_j}$$

**Average execution time**

- Arithmetic mean of the absolute execution times of the programs of the suite
- It is reliable when the workload is distributed uniformly over multiple applications

$$\overline{T_e} = \frac{1}{N} \sum_{j=1}^{N} T_{e_j}$$

**Weighted execution time**

- If the workload is not uniform, it is more trustworthy to keep into consideration the relative frequency $w_j$ with which each program is executed

$$T_{e_w} = \sum_{j=1}^{N} w_j T_{e_j}$$

**Equal time weighting**

- All programs of the suite are executed for the same amount of time

$$w_j = \frac{1}{T_{e_j} \cdot \sum_{i=1}^{N} \frac{1}{T_{e_i}}}$$

# Aside: Equal Time Weighting

**Equal time weighting**

- All programs of the suite are executed for the same amount of time

**Weight inversely proportional to the execution time**

- $z_j = \frac{1}{T_j}$

**Sum of all the weights must be 1**

- Divide all weights by their sum
- $w_j = \frac{z_j}{\sum_{i=1}^{N} z_i} = \frac{1}{T_j} \cdot \frac{1}{\sum_{i=1}^{N} \frac{1}{T_i}}$

$$w_j = \frac{1}{T_{e_j} \cdot \sum_{i=1}^{N} \frac{1}{T_{e_i}}}$$

# EXAMPLE

| | Programs | | | Weightings | | |
|---|---|---|---|---|---|---|
| | A | B | C | W(1) | W(2) | W(3) |
| Program P1 (secs) | 1.00 | 10.00 | 20.00 | 0.50 | 0.909 | 0.999 |
| Program P2 (secs) | 1000.00 | 100.00 | 20.00 | 0.50 | 0.091 | 0.001 |

| | | | |
|---|---|---|---|
| Arithmetic mean:W(1) | 500.50 | 55.00 | 20.00 |
| Arithmetic mean:W(2) | 91.91 | 18.19 | 20.00 |
| Arithmetic mean:W(3) | 2.00 | 10.09 | 20.00 |

**W(1)**

- equally weighs the programs, resulting in the classic arithmetic mean

**W(2)**

- weighs the programs in inverse proportion to the execution times on machine **B**

**W(3)**

- weighs the programs in inverse proportion to the execution times on machine **A**

**The second and third weightings "normalize" the weights to the execution times of programs running on that machine, so that the running time will be spent evenly between each program for that machine**

# Some hints on embedded benchmarks

**EEMBC suite consists mostly of kernels (rather than of full applications) which are particularly important in different classes of embedded fields**

| Benchmark Type | # of this type | Example benchmarks |
|---|---|---|
| Automotive/industrial | 16 | 6 microbenchmarks (arithmetic operations, pointer chasing, memory performance, matrix arithmetic, table lookup, bit manipulation), 5 automobile control benchmarks, and 5 filter or FFT benchmarks. |
| Consumer | 5 | 5 multimedia benchmarks (JPEG compress/decompress, filtering, and RGB conversions). |
| Networking | 3 | Shortest path calculation, IP routing, and packet flow operations. |
| Office automation | 4 | Graphics and text benchmarks (Bezier curve calculation, dithering, image rotation, text processing). |
| Telecommunications | 6 | Filtering and DSP benchmarks (autocorrelation, FFT, decoder, and encoder) |

**In EEMBC, hand-coding is allowed. In particular, EEMBC allows assembly language coding of its benchmarks**

DESIGN A NEW SYSTEM

IMPROVE AN EXISTING SYSTEM

# QUANTITATIVE PRINCIPLES FOR THE DESIGN OF COMPUTING SYSTEMS

# General issues in designing a new system

**Designing a new processing system or architecture is a multi-variable, multi-objective optimization problem**

- Usually returns many different solutions in terms of *execution time* ($T$), *cost* ($C$) and *power consumption* ($P$)

**The optimal solution results from the tradeoff of different needs**

- A good solution can be determined by finding the system configuration that minimizes a cost function depending on the variables $T, C, P$
- These in turn are functions of several architectural parameters such as: ISA, memory size and type, number of functional units (ALU, MUL,…), etc.

**Examples of design problems**

- Desktops: Find the configuration $X_i$ such that

$$Y_1(X_i) = T(X_i) \cdot C(X_i) \text{ is min. and } P(X_i) \leq P_{max}$$

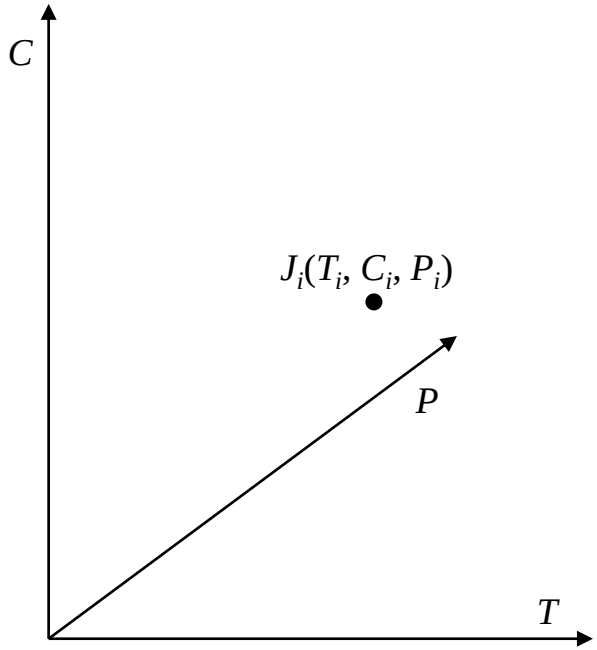
- Embedded: Find the configuration $X_i$ such that

$$Y_2(X_i) = C(X_i) \cdot P(X_i) \text{ is min. and } T(X_i) \leq T_{max}$$

# The design space

**The design space is a *n*-dimensional space consisting of all the possible tuples of design parameters (configurations)**

- The total number $N_c$ of possible system configurations is usually very large
- For each configuration $X_i$ , $i = 1, \ldots, N_c$ a triple $(T_i, C_i, P_i)$ exists
- This triple identifies a point $J_i$ (design point) in the 3D space $(T, C, P)$



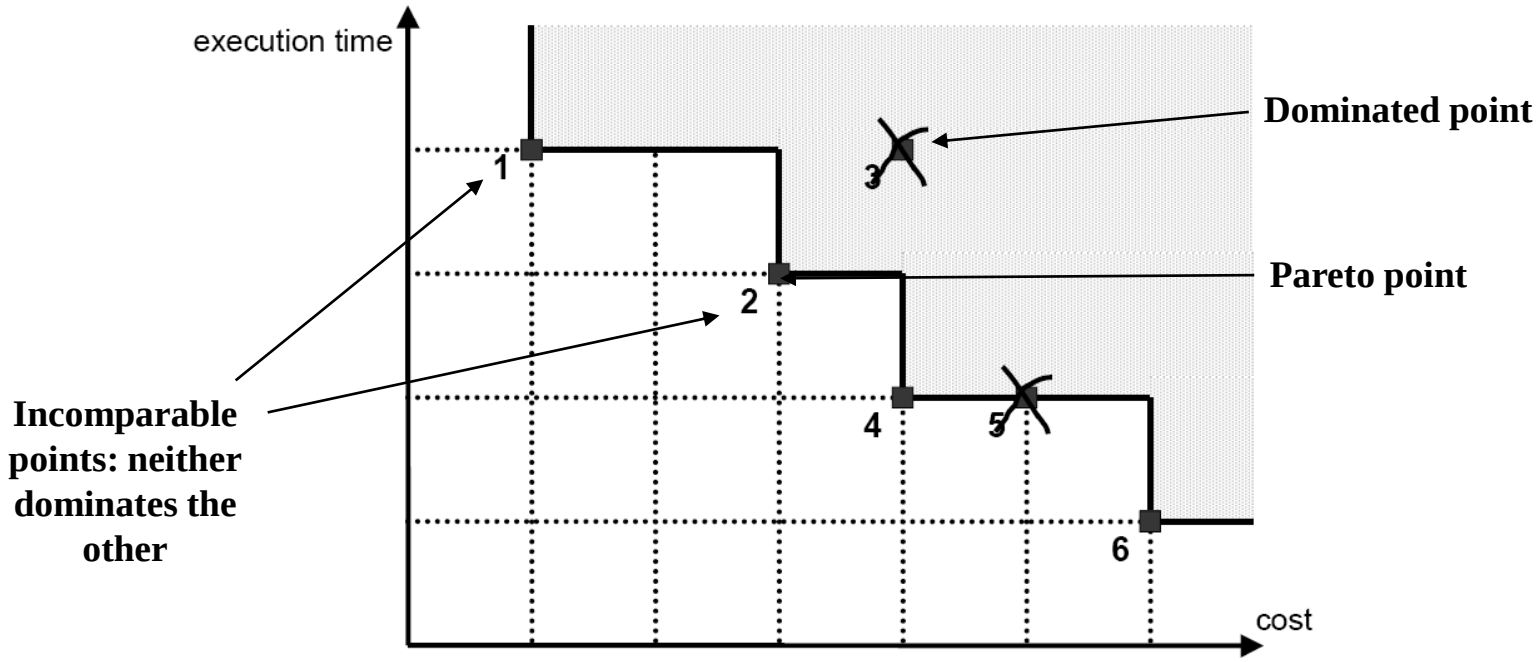
# Design points and Pareto points

**Definition: A design point $J_k$ is *dominated* by $J_i$, if $J_i$ is:**

- better or equal than $J_k$ in all criteria and
- better in at least one criterion.

**Definition: A point is *Pareto-optimal* or a *Pareto-point*, if it is not dominated.**

**Example: tradeoff between cost and execution time**



# Design space exploration (DSE)

**The task of looking for the Pareto points within the design space is called *design space exploration***

**Design space exploration can be**

- exhaustive (impractical due to the large number of points)
- based on optimization algorithms (greedy search, mathematical programming, genetic algorithms, etc.)

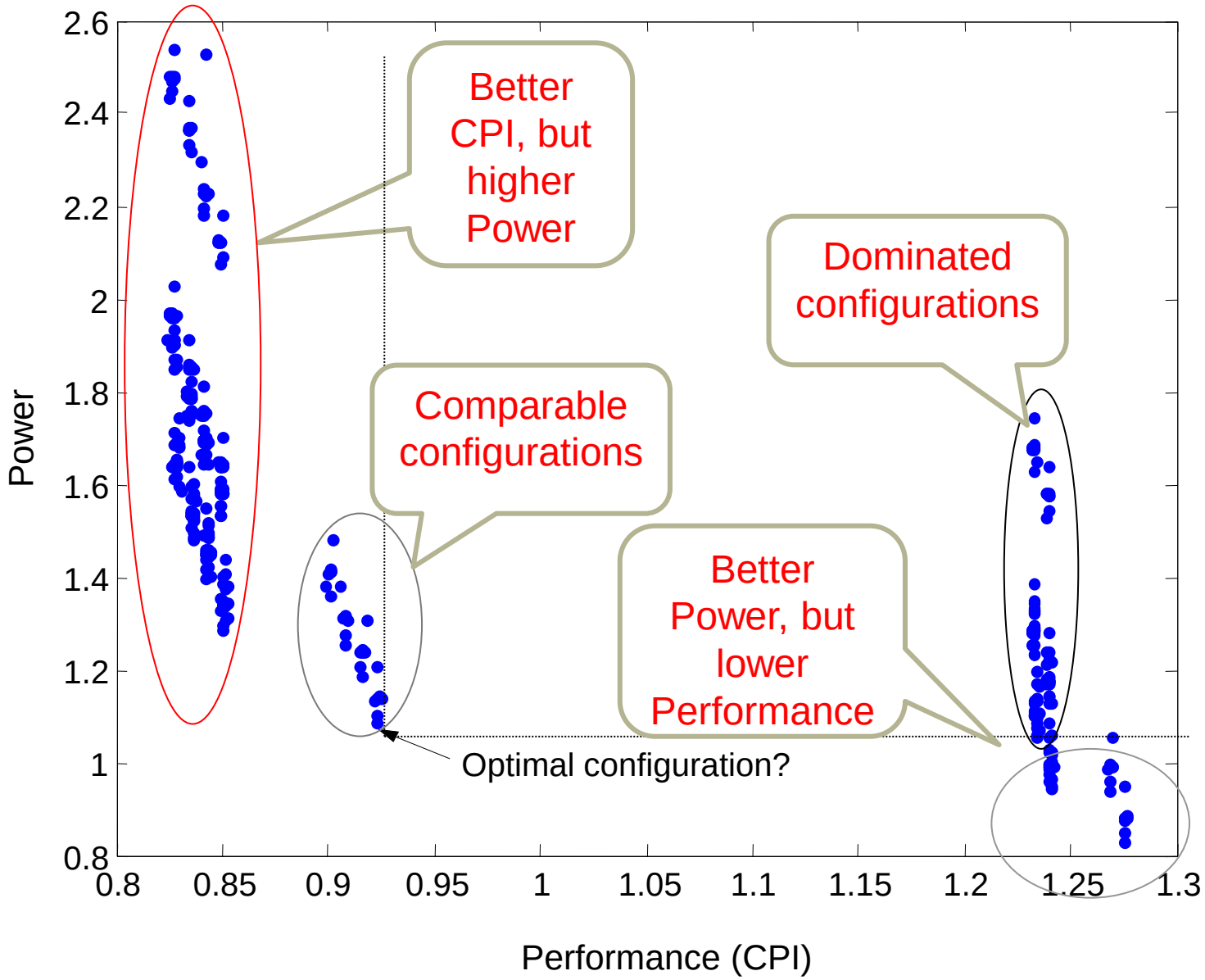
**Example: Find the most effective embedded processor in terms of power and performance to run an encryption algorithm (e.g., Pegwit)**

**An example of design space could be the following:**

- Different sizes in L1 Instruction Cache: size {512, 1024,…} bytes;
- Different sizes in L1 Data Cache: size {512, 1024,…} bytes;
- Different sizes in L2 Unified Cache: size {2048, 4096,…} bytes;
- Different number of ALUs: {2, 4, 8};
- Different number HW Multipliers: {1, 2,…}

**hundreds of configurations**

# Example of exhaustive DSE



# Summary

**Performance**

- Execution time
- CPU time (CPI, Instruction Count, Clock frequency)

**Multi-objective optimization**

- Dominance of solution
- Pareto front

# Improving an existing system

**Assume we add an enhancement E to a given system or architecture**

**The *speedup* is the relative performance improvement due to E**

- How faster is the architecture with E?

$$\text{Speedup}(\text{E}) = \frac{\text{ExTime without E}}{\text{ExTime with E}} = \frac{\text{Performance with E}}{\text{Performance without E}}$$

**Example**

- ExTime w/o E = 14 s
- ExTime w/ E = 8 s
- Speedup( E ) = 14 / 8 = 1.75

# Improving an existing system

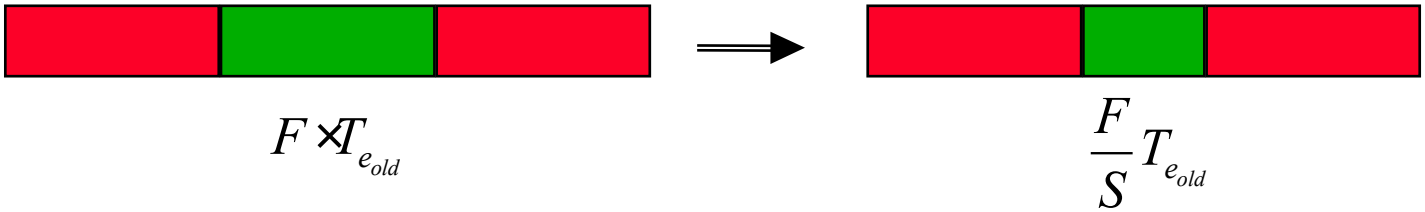
**The overall Speedup (E) depends on:**

- The fraction of computation ($F \leq 1$) for which the enhancement $E$ is used (green area)
- The speedup ($S > 1$) which we could achieve if $E$ was used 100% of execution time

**The execution time of a certain program after *E* is added is:**

$$T_{e_{new}} = T_{e_{old}} - F \cdot T_{e_{old}} + \frac{F}{S} T_{e_{old}}$$

$$T_{e_{new}} = T_{e_{old}} \left[ (1 - F) + \frac{F}{S} \right]$$



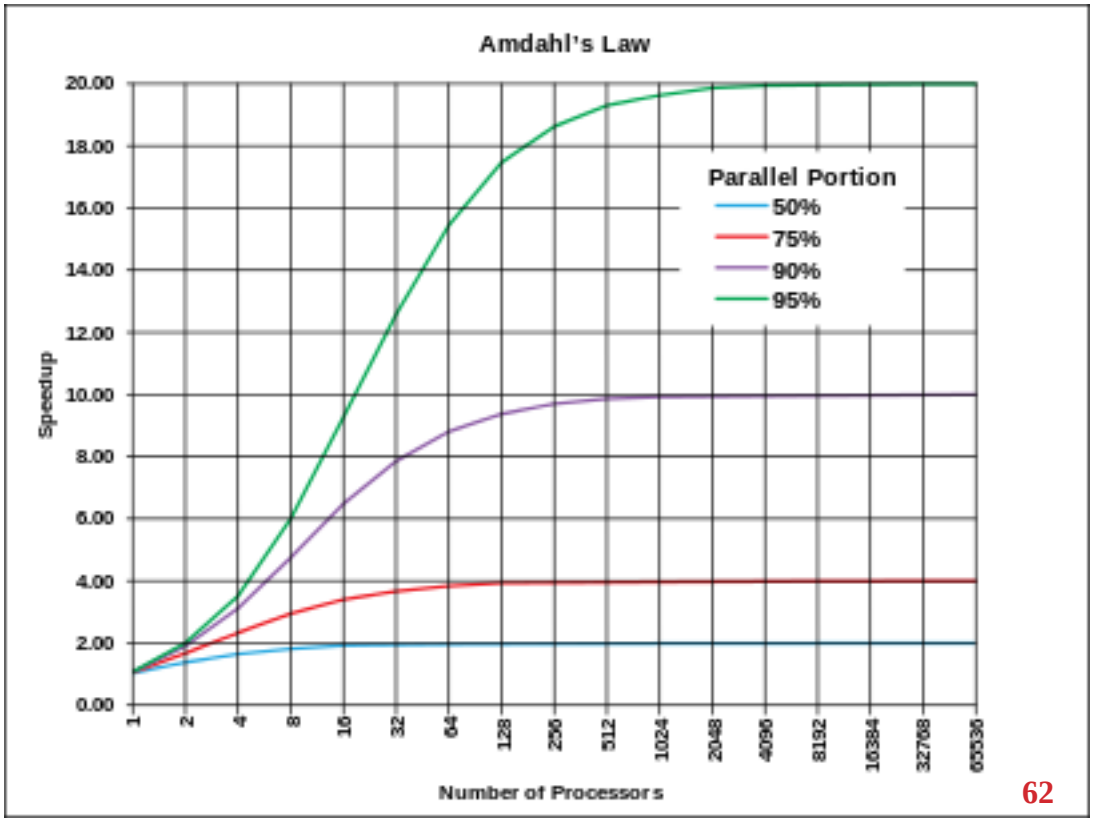
# Amdahl's Law

**The overall speedup is:**

$$S_{overall} = \frac{T_{e_{old}}}{T_{e_{new}}} = \frac{1}{(1-F) + \frac{F}{S}} \xrightarrow{S \to \infty} \frac{1}{1-F}$$

**Amdahl's Law**

- **The performance improvement to be gained from using some faster mode of execution is limited by the fraction of time the faster mode can be used**



# Amdahl's Law

$$S_{overall} = \frac{T_{e_{old}}}{T_{e_{new}}} = \frac{1}{(1-F) + \frac{F}{S}} \xrightarrow{S \to \infty} \frac{1}{1-F}$$

**Corollary 1**

- Make the common case (*F* is largest) as fast as possible

**Corollary 2 (Law of diminishing returns)**

- The speedup due to additional enhancements affecting the same fraction of computation *F* tends to decrease as the number of enhancements grows
  - Why? Because *F* becomes progressively smaller!
- It could be unprofitable to add an excessive number of enhancements affecting only the same fraction *F* of computation

# EXAMPLE

**A transformation required in graphics engines is square root**

- Suppose FP square root (FPSQR) is responsible for 20% of the execution time of a graphics benchmark

**1° proposal**

- enhance the FPSQR hardware and speed up this operation by a factor of 10

**2° proposal**

- make all FP instructions faster by a factor of 1.6 (FP instructions are responsible for a total of 50% of the execution time for the application)

**Which solution is better?**

$$\text{Speedup}_{\text{FPSQR}} = \frac{1}{(1-0.2)+\frac{0.2}{10}} = \frac{1}{0.82} = 1.22$$

$$\text{Speedup}_{\text{FP}} = \frac{1}{(1-0.5)+\frac{0.5}{1.6}} = \frac{1}{0.8125} = 1.23$$

**Improving the performance of the FP operations overall is slightly better because of the higher execution frequency**

# Summary

**Several metrics must be considered in design**

- Performance (execution time)
- Cost
- Power consumption

**Metrics sometimes hard to estimate**

- Need meaningful metrics for comparison

**Design is a multi-objective optimization process**

- Optimal design is the result of several trade-offs

**Must choose what to improve carefully**

- Large enhancements not useful if not used
- Beware of the law of diminishing returns

# Main references

H. Hennessy, D. A. Patterson, "Computer Architecture: a quantitative approach," Morgan Kaufmann, 2002, Chapter 1

D. J. Liljia, “Measuring Computer Performance – A practitioner’s guide”, Cambridge University Press, 2000, Chapters 1-3